

RoboCall AI

How the system works, end to end

Scripted outbound voice calls over Twilio — FastAPI server, background dispatcher, SQLite store.

1. What the system does

You define a **campaign**: a message, a voice, a contact list and a schedule. The system turns that into individual call tasks, screens each one against your opt-out list and the recipient's local calling hours, then hands the allowed ones to Twilio at a controlled rate. Twilio dials the number and reads the script with a neural text-to-speech voice. The outcome of every call is written back to the database.

This is **phase 1: play-and-read**. The system dials, speaks, and hangs up. It does not listen, transcribe, or hold a conversation. The only recipient input it can accept is a keypress to opt out, and only in webhook mode.

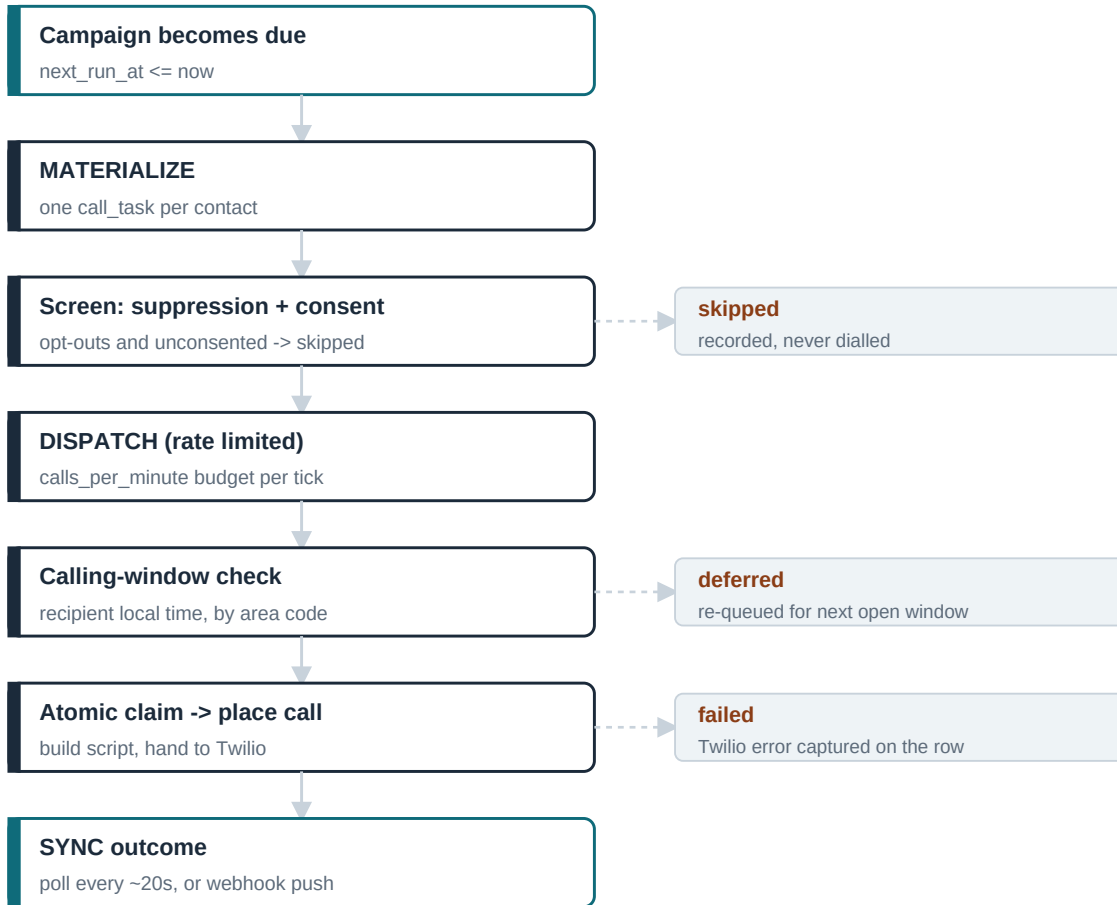
The dispatcher runs as a daemon thread *inside* the web server process, so calls continue going out whether or not a browser tab is open. Closing the dashboard does not stop a campaign; stopping the server does.

2. The moving parts

File	Role
server.py	FastAPI. Serves the dashboard, the JSON API, and the three Twilio webhook endpoints. Starts and stops the dispatcher thread on app lifespan.
dispatcher.py	The call engine. A daemon thread that ticks every 5 seconds through materialize, dispatch, and sync.
telephony.py	The Twilio provider. Builds TwiML, places calls, fetches call status, and decides direct vs webhook delivery.
compliance.py	Guardrails. Number normalisation, area-code to timezone mapping, calling-window arithmetic, and required script language.
config.py	Settings resolution. Database values win over environment variables, so the dashboard is the source of truth once used.
db.py	SQLite persistence, schema creation, and lightweight column migration.
scriptwriter.py	Optional Claude-backed script drafting for the 'Draft with AI' button.
web/	The dashboard — plain HTML, CSS and JS. No build step.

3. The lifecycle of a call

Every call travels the same path. The dotted branches are the three ways a task leaves the pipeline without ever reaching a recipient.



Separating **materialize** from **dispatch** is what makes pacing and quiet-hours deferral possible. A task that cannot be called right now is left in the queue with a later *scheduled_for* timestamp, rather than being dropped or dialled anyway.

4. Stage by stage

Stage 0 — Configuration

Every setting resolves in the same order: the value stored in the database wins; if it is empty, the environment variable is used; if that is empty, a built-in default applies. Twilio credentials, the caller ID, the calling window, the pace, and the business identity all flow through this. The Twilio client itself is cached on the credential tuple, so changing a credential in Settings transparently rebuilds it on the next call.

Stage 1 — Creating a campaign

A campaign carries a name, the message, a voice, an answering-machine mode, a frequency (once, hourly, daily or weekly), a wall-clock time, a timezone, and a consent requirement. Contacts are attached to the campaign, each with a phone number and a consent flag. The script is validated on save. Daily and weekly campaigns are anchored to wall-clock time in the campaign's own timezone, so a 10:00 campaign stays at 10:00 across a daylight-saving change.

Stage 2 — Materialize

On each tick the dispatcher looks for active campaigns whose *next_run_at* has passed. For each one it writes a *call_tasks* row per contact and rolls the campaign's next run forward. Two screens are applied at this moment:

- A number on the **suppression list** is written as *skipped / suppressed* and never dialled.
- If the campaign requires consent and the contact has not given it, the task is written as *skipped / no-consent*.
- A campaign with an empty message queues nothing at all and logs an error — otherwise it would dial the whole list and read the stock test script at them.

Skipped tasks are still recorded. You get a complete audit trail of who was *not* called and why, which is the record you need if an opt-out is ever disputed.

Stage 3 — Dispatch

Pending tasks are pulled oldest-first, limited to a per-tick budget derived from your *calls per minute* setting. Each task then passes three gates before it is dialled:

- **Re-check suppression.** Someone may have opted out in the minutes since the task was queued.
- **Calling window.** The recipient's timezone is inferred from their area code and their local time is checked against the window. Outside it, the task is set to *deferred* with *scheduled_for* moved to the next opening — it is re-queued, not discarded. If the area code is unknown, every candidate zone must be inside the window before the call goes out.
- **Atomic claim.** A conditional UPDATE takes exclusive ownership of the row. Dispatch runs on the dispatcher thread *and* can be invoked from a request handler (the test-call button), so two callers really can reach the same task. The claim only matches while the task is still unclaimed, so exactly one wins and nobody is dialled twice.

Only then is the script assembled and the call handed to Twilio. If the provider raises, the exception is caught and recorded as one failed call — it does not abandon the claimed row in *dialing* forever, and it does not abort the rest of the batch.

Stage 4 — The call

The message is wrapped in the language a pre-recorded call is required to carry, rendered as TwiML, and spoken by the selected voice. Answering-machine handling is chosen per campaign: talk immediately, wait for the greeting to finish before talking, or hang up on machines entirely (which needs webhook mode).

Stage 5 — Sync

Roughly every 20 seconds the dispatcher polls Twilio for calls still in flight and records the status, who or what answered, and the duration. In webhook mode Twilio pushes these instead and the poll becomes a backstop. A call that never produces a final status is closed out as *unknown* after two hours rather than sitting in *dialing* indefinitely.

5. Two delivery modes

Which mode is used is decided per call, purely by whether a public base URL is configured. This is the single most consequential setting in the system.

	Direct mode (default)	Webhook mode
Trigger	No PUBLIC_BASE_URL	PUBLIC_BASE_URL set to a public HTTPS origin
How Twilio gets the script	The whole TwiML document rides along on the call-creation request.	Twilio fetches the TwiML back from this server.
Needs to be reachable from the internet?	No — runs on a laptop.	Yes.
How outcomes arrive	Polled from the Twilio API.	Pushed as status callbacks.
Press 9 to opt out	Not available.	Available.
Hang up on voicemail	Not available.	Available.

Webhook requests are verified against Twilio's request signature before they are acted on, so a forged callback cannot mark calls complete or add entries to your opt-out list.

6. Compliance guardrails

These are built into the pipeline rather than left to the operator to remember.

- **Caller identification.** The business name is prepended to every script automatically, unless the message already contains it.
- **Opt-out instructions.** A removal instruction is appended to every script. The callback number is spelled digit by digit so text-to-speech reads it as a phone number rather than as a quantity.
- **Calling hours.** Enforced in the *recipient's* local time, derived from their area code — not yours. The federal TCPA limit is 08:00 to 21:00; the shipped default is 09:00 to 20:00, because several states are stricter.
- **Suppression list.** Checked twice — once when the task is queued, once immediately before dialling.
- **Consent.** Campaigns can require a per-contact consent flag and will skip anyone lacking it.
- **No retries.** Phase 1 does not redial a busy or unanswered number.

7. What a call task's state means

State	Meaning
pending	Queued and waiting for its turn.
deferred	Outside the recipient's calling window. Re-queued for the next opening.
dialing	Claimed and handed to Twilio; awaiting a final status.
skipped	Never dialled — suppressed, or no consent.
done	Finished. The status column carries the outcome: completed, busy, no-answer, canceled, failed, or unknown.

8. Where things are stored

Table	Holds
campaigns	Message, voice, schedule, timezone, consent requirement, run bookkeeping.
contacts	Phone numbers per campaign, each with a consent flag.
call_tasks	One row per attempted call: state, status, Twilio SID, duration, error, token.
suppression	The opt-out list.
events	The activity log.
settings	Dashboard-entered configuration, which overrides the environment.

The database lives at %LOCALAPPDATA%\RoboCallAI\robocall.db — deliberately outside the Dropbox folder. Dropbox replaces files it is syncing, including a SQLite database an app currently has open, which silently destroys campaigns and logs. Override with ROBOCALL_DB, but keep it off any synced drive.

9. Current status and what blocks a deployment

The pipeline above is implemented and covered by tests: 74 happy-path checks against a fake Twilio, plus 129 adversarial checks covering concurrency, Twilio errors and forged webhooks. The following are the open items between here and a working public deployment.

Configuration gaps

- **No caller ID is set.** TWILIO_FROM_NUMBER is empty, so no call can be placed. It must be a voice-capable number purchased from Twilio, or a verified caller ID.
- **A Google Voice number cannot serve as the caller ID.** Twilio requires a number you own through Twilio or have verified with it. The Google number is stored and displayed by the retired prototype but is never used to place a call, and the current server does not read it at all.
- **The Twilio API key and secret are unused.** Authentication is by account SID and auth token only. The key pair is harmless but does nothing.
- **The test destination number is empty.** On a Twilio trial account it must also be verified in the Twilio console before Twilio will connect a call to it.

Deployment blockers

- **There is no authentication on any route.** Anyone who finds the URL can queue calls against your Twilio balance and read your contact lists. Acceptable on localhost; not on the public internet.
- **SQLite on an ephemeral filesystem is lost on every restart.** That takes the opt-out list with it, which is a legal exposure rather than merely a data loss. Attach a persistent volume and point ROBOCALL_DB at it, or move to Postgres.
- **This is not a Streamlit application.** It is FastAPI plus a static frontend, served by uvicorn. Streamlit Community Cloud can only run `streamlit run <file>`, and it sleeps idle apps — which would stop the dispatcher thread mid-campaign. Use a host that runs a long-lived process.

Run it locally

```
python -m uvicorn server:app --port 8000
```

Then open <http://localhost:8000>.